27 Oct 2026 Jebel-Quant/lasso 366e3fb

cvxcla: The Critical Line Algorithm with an Operator Interface in Python

Thomas Schmelzer
Jebel Quant Research

Abstract

cvxcla is an open-source Python implementation of Markowitz's Critical Line Algorithm (CLA), which returns the *entire* mean–variance efficient frontier as a finite list of turning points, with no discretisation error. The turning points are Markowitz's; the package adds an interface to the covariance and a wider range of constraints. The algorithm reaches the covariance only through a small operator protocol, so a dense matrix, a diagonal-plus-low-rank risk model, a data matrix, or a user-written backend can be exchanged without touching the loop. With 2560 assets and ten factors the factor backend traces the same frontier about 35 times faster than the dense one. Box bounds, linear equalities and inequalities, and a cap on gross exposure are supported. The walk provably terminates under standard nondegeneracy assumptions, every trace is checked by an independent Karush–Kuhn–Tucker certificate, and inputs outside the supported domain stop with an error that names the cause.

Keywords: Critical Line Algorithm, mean–variance optimization, efficient frontier, parametric quadratic programming, active-set method, factor covariance models, Python.

1. Introduction

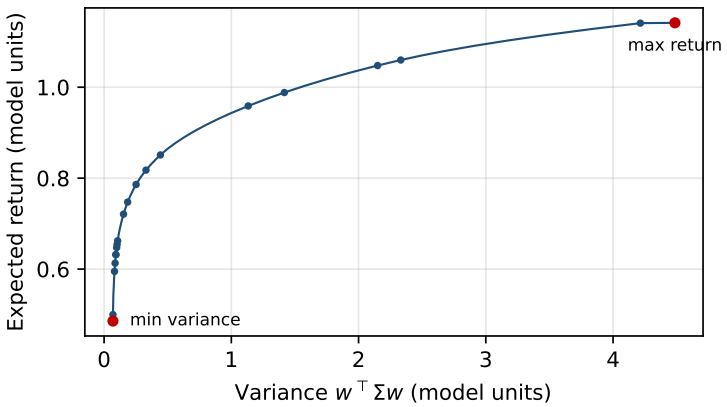


Figure 1: The efficient frontier of a 20-asset long-only problem, traced exactly by **cvxcla** as 20 turning points, plotted as return against variance (`make_figures.py frontier`).

The mean–variance portfolio selection problem (Markowitz 1952) asks for the allocation of

capital across n assets that minimises risk for a given level of expected return; sweeping the return level produces a one-parameter family of optimal portfolios whose risk–return image is the *efficient frontier*. The Critical Line Algorithm (CLA) of Markowitz (1956, 1959) represents this frontier *exactly and in full*: it returns a finite list of *turning points* between which the frontier is described in closed form, so no discretisation or repeated re-optimisation is required. The canonical description and a reference implementation are due to Markowitz and Todd (2000); Perold (1984), Stein, Branke, and Schmeck (2008), and Hirschberger, Qi, and Steuer (2010) develop large-scale and active-set variants, Niedermayer and Niedermayer (2010) a careful fast implementation, Bailey and López de Prado (2013) a widely used open-source one, and Markowitz, Starer, Fram, and Gerber (2021) a recent review extending the method to mean–semivariance. In optimisation terms the CLA solves a parametric quadratic program. Wolfe (1959) adapted the simplex method to quadratic programs, Best (1996) gave a parametric active-set algorithm that Best (2010) applies to portfolios, and **qpOASES** (Ferreau, Kirches, Potschka, Bock, and Diehl 2014) is a general parametric QP solver.

Classical CLA implementations take the covariance as a dense matrix. For a large universe it usually has more structure than that: multi-factor risk models, which generalise the single-index model (Sharpe 1963), write it as a diagonal matrix plus a low-rank term. The large-scale methods cited above exploit structure through incremental factorisations, sparsity, or pivoting tied to an explicit matrix. **cvxcla** computes the same turning points as any CLA; its contribution is an *interface* that makes the loop independent of how the covariance is stored, so the structure becomes a property of an exchangeable backend (Appendix C). The mathematics connecting the CLA to the constrained LASSO, and its history, are the subject of a companion paper (Schmelzer and Hastie 2026). Schmelzer (2026b) shows how to obtain the efficient frontier from a LASSO solver, and Schmelzer (2026a) goes the other way, computing constrained LASSO paths with **cvxcla**; this paper documents the software.

Contributions.

1. **An operator interface.** The turning-point loop reaches the covariance through a five-member protocol (§5). A diagonal-plus-low-rank backend solves the free block through the Woodbury identity, never forms an $n \times n$ matrix, and traces the same turning points about $40\times$ faster than the dense backend at $n = 2560$, in under 1 MiB of working memory against 100 MiB (§7).
2. **General linear constraints.** Box bounds combine with any linear equalities $Aw = b$ and inequalities $Gw \leq h$ (§4.3) and a gross-exposure cap $\|w\|_1 \leq c$ (§4.4).
3. **A tested implementation.** A least-index tie-break makes the trace deterministic and provably finite under standard assumptions (§4.5); an independent KKT certificate validates every trace (§7.1); inputs outside the supported domain are declined with an error that names the cause (§6.3); and one script regenerates every figure and number.

Table 1 sets these capabilities against the software a user would otherwise reach for.

We do *not* claim the fastest solve: on small problems **qpOASES** is faster (§7.6), and large dense problems admit faster incremental factorisations (§7.3).

The package (Schmelzer 2026c)[1] is built around the class `CLA` in `src/cvxcla/cla.py`, sup-

[1] `github.com/cvxgrp/cvxcla`

Software	Output	Covariance	Constraints	Gross cap
PyPortfolioOpt `CLA`	turning points, some lost	dense matrix	box bounds, one budget row	no
QP over a λ grid	one portfolio per grid point	as the solver accepts it	any convex	reformulated
qpOASES via **CasADi**	path followed, endpoint returned	matrix	general linear	reformulated
cvxcla	every turning point	any operator	box bounds, $Aw = b$, $Gw \le h$	native lift

Table 1: What each route to the mean–variance frontier delivers, as used in §7: the QP over a λ grid is described in §7.5, **qpOASES** in §7.6, **cvxcla**'s covariance operators in §5 and its lift of the gross cap in §4.4. "Gross cap" is $\|w\|_1 \le c$. "Path followed, endpoint returned" means **qpOASES** passes the same turning points but does not expose them through **CasADi**. The termination guarantee (§4.5) and the KKT certificate (§7.1) are properties of **cvxcla**'s trace and are not compared here.

ported by `first.py` (the starting portfolio), `operators/` (the covariance backends), and `types.py` (the turning-point and frontier data structures). The notation follows the code.

Sections 2–4 describe the algorithm, §5 the operator interface, §6 the user interface and §7 the experiments; details and the termination proof are in the appendices.

2. Problem formulation

Let $w \in \mathbb{R}^n$ be the portfolio weights, $\mu \in \mathbb{R}^n$ the expected returns, $\Sigma \in \mathbb{R}^{n\times n}$ the (symmetric, positive semidefinite) covariance, $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ the equality constraints, $G \in \mathbb{R}^{p\times n}$, $h \in \mathbb{R}^p$ the inequality constraints, and ℓ, u the bounds. The data μ, Σ, A, b, G, h must be finite. A bound may be infinite, an unbounded box; the first turning point is then found by the linear program of §3, which refuses a problem whose expected return is unbounded. The implementation traces the frontier through the *return-parametrised* quadratic program

$$\min_{w\in\mathbb{R}^n} \ \tfrac{1}{2}\, w^{\mathsf{T}}\Sigma\, w - \lambda\, \mu^{\mathsf{T}} w \quad \text{subject to} \quad Aw = b, \ \ Gw \le h, \ \ \ell \le w \le u, \tag{1}$$

where $\lambda \ge 0$ weights expected return against variance. The canonical instance has the single budget row $\mathbf{1}^{\mathsf{T}} w = 1$; general A and G (group or sector caps, a $\ge$ row negated) are handled in §3 and §4.3. As λ decreases from $+\infty$ to 0, the optimiser moves from the maximum-return portfolio to the global minimum-variance portfolio, and its locus is the efficient frontier (Figure 1).

Optimality conditions. At fixed λ the program is convex, so the KKT conditions are necessary and sufficient. Partition the assets into a *free* set $\mathcal{F}$ (weights strictly inside their bounds) and a *blocked* set $\mathcal{B} = \mathcal{F}^c$ (weights at ℓ_i or u_i), and let $\mathcal{S} \subseteq \{1, \dots, p\}$ index the *active* inequality rows. For the free assets

$$\begin{aligned} \Sigma_{\mathcal{F}\mathcal{F}}\, w_{\mathcal{F}} + \Sigma_{\mathcal{F}\mathcal{B}}\, w_{\mathcal{B}} + A_{\mathcal{F}}^{\mathsf{T}}\nu + G_{\mathcal{S}\mathcal{F}}^{\mathsf{T}}\eta &= \lambda\, \mu_{\mathcal{F}}, \\ A_{\mathcal{F}}\, w_{\mathcal{F}} + A_{\mathcal{B}}\, w_{\mathcal{B}} = b, \qquad G_{\mathcal{S}\mathcal{F}}\, w_{\mathcal{F}} + G_{\mathcal{S}\mathcal{B}}\, w_{\mathcal{B}} &= h_{\mathcal{S}}, \end{aligned} \tag{2}$$

with multipliers $\nu \in \mathbb{R}^m$ and $\eta \in \mathbb{R}^{|\mathcal{S}|}_{\geq 0}$. The blocked weights are known, so (2) is linear in $(w_{\mathcal{F}}, \nu, \eta)$, and an active inequality row enters exactly as an equality row: stacking $C = [\,A;\, G_{\mathcal{S}}\,]$ recovers the equality-only form.

The affine segment. While the partition is fixed, the system matrix is constant and the right-hand side affine in λ, so

$$w(\lambda) = \alpha + \lambda\,\beta, \tag{3}$$

with α, β constant on the segment ($\beta_{\mathcal{B}} = 0$, $\alpha_{\mathcal{B}} = w_{\mathcal{B}}$); the code calls them `r_alpha` and `r_beta`. This is the *critical line*. A *turning point* is a value of λ at which the partition must change.

3. The first turning point

At $\lambda = +\infty$ variance is irrelevant and (1) reduces to earning the most expected return the constraints allow. For a single all-ones budget row $\mathbf{1}^{\mathsf{T}} w = b$, `init_algo` (`first.py`) computes this greedily: start every weight at ℓ, sort the assets by descending μ, and raise each towards u_i until the budget is met (within a tolerance of 10^{-12}; without it, round-off can skip the asset that should end strictly inside its box). The last asset touched absorbs the residual and is the single *free* asset; all others are blocked. The same fill serves leverage ($b > 1$) and dollar-neutral books ($b = 0$, negative ℓ). The last asset is marked free even when it lands exactly on a bound, so the reduced KKT system is non-singular from the outset, a case the review of Markowitz *et al.* (2021) does not address.

For a general equality system (weighted rows, or $m > 1$ rows such as sector or factor neutrality) or any G, the maximum-return vertex is the linear program $\max_w \mu^{\mathsf{T}} w$ subject to the constraints, which **cvxcla** solves with the HiGHS simplex through `scipy.optimize.linprog`; the free set is read off as the assets strictly inside their bounds. Only this first step differs; the loop that follows is written for a general A throughout.

Tied maximum returns. The maximum-return set need not be a single vertex. When two or more assets share the largest expected return, say because they were given the same forecast, the linear program has a whole face of optima, and the greedy fill or the simplex returns one of its vertices. The frontier does not start at an arbitrary one. For λ just below $+\infty$ the optimiser of (1) is the point of that face where the quadratic term $\frac{1}{2} w^{\mathsf{T}} \Sigma w$ is smallest, and the path leaves from there. Since release 2.6.0, **cvxcla** starts at that point, a small quadratic program over the face, and its free set is the assets strictly inside their bounds there. Earlier releases started at the vertex the solver returned. Release 2.3.3, used for the first version of this paper, then traced a frontier that ended but was not efficient; that version reported such problems as completed (Appendix B). Releases 2.4.2 to 2.5.0 declined them instead.

The vertex itself can be degenerate, with every weight on a bound or a duplicated tight row. Its partition is then read from a vertex of the linear program's optimal dual set, which keeps it dual feasible and starts the first segment at the vertex itself (Appendix A.3). Redundant equality rows, a repeated budget say, are dropped before the trace starts; a vertex is refused with a `DegenerateProblemError` (§6.3) only when its optimal dual set has no vertex.

4. The turning-point iteration

From the first turning point the algorithm repeatedly (i) solves the reduced KKT system for the affine segment (3); (ii) finds the largest λ below the current one at which an *event* occurs; (iii) flips the status of the single coordinate responsible; and (iv) records the new turning point, stopping at $\lambda = 0$.

4.1. Solving the reduced KKT system by block elimination

Write $C_{\mathcal{F}}$ for the active constraint rows restricted to the free set ($A_{\mathcal{F}}$ when $\mathcal{S} = \varnothing$; §4.3) and c for their right-hand side. The reduced system is the saddle-point problem

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & C_{\mathcal{F}}^{\mathsf{T}} \\ C_{\mathcal{F}} & 0 \end{bmatrix} \begin{bmatrix} w_{\mathcal{F}} \\ \nu \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}, \tag{4}$$

solved for two right-hand sides at once, the "α system" $r_1^{\alpha} = -\Sigma_{\mathcal{F}\mathcal{B}} w_{\mathcal{B}}$, $r_2^{\alpha} = c - C_{\mathcal{B}} w_{\mathcal{B}}$, and the "$\beta$ system" $r_1^{\beta} = \mu_{\mathcal{F}}$, $r_2^{\beta} = 0$. Rather than factor the full saddle matrix, the code eliminates the free block. A single multi-column solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ gives $Y = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} C_{\mathcal{F}}^{\mathsf{T}}$ and $z^{\alpha,\beta} = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^{\alpha,\beta}$. The multipliers come from the small Schur complement $S = C_{\mathcal{F}} Y$. With $n_{\mathcal{F}}$ free assets and $m_c = m + |\mathcal{S}|$ active rows, $C_{\mathcal{F}} \in \mathbb{R}^{m_c \times n_{\mathcal{F}}}$, $Y \in \mathbb{R}^{n_{\mathcal{F}} \times m_c}$ and $S \in \mathbb{R}^{m_c \times m_c}$,

$$\nu^{\alpha} = S^{-1}(C_{\mathcal{F}} z^{\alpha} - r_2^{\alpha}), \qquad \nu^{\beta} = S^{-1} C_{\mathcal{F}} z^{\beta}, \tag{5}$$

and back-substitution gives

$$\alpha_{\mathcal{F}} = z^{\alpha} - Y \nu^{\alpha}, \qquad \beta_{\mathcal{F}} = z^{\beta} - Y \nu^{\beta}. \tag{6}$$

The dominant cost is the solve against $\Sigma_{\mathcal{F}\mathcal{F}}$; for the plain budget S is a scalar. With $1 < m_c \le n_{\mathcal{F}}$ the rows are first replaced by an orthonormal basis of their span, $C_{\mathcal{F}}^{\mathsf{T}} = QR$, and the system is solved with Q in place of $C_{\mathcal{F}}^{\mathsf{T}}$, the target $R^{-\mathsf{T}} r_2$, and the multipliers recovered by a triangular solve with R. The condition number of $C_{\mathcal{F}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} C_{\mathcal{F}}^{\mathsf{T}}$ can reach $\operatorname{cond}(\Sigma_{\mathcal{F}\mathcal{F}}) \operatorname{cond}(C_{\mathcal{F}})^2$, while $Q^{\mathsf{T}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} Q$ is no worse conditioned than $\Sigma_{\mathcal{F}\mathcal{F}}$, so nearly dependent active rows lose accuracy in proportion to $\operatorname{cond}(C_{\mathcal{F}})$, the sensitivity of the constraint data, and not to its square. Solving only for the free block keeps every step a dense solve of size $|\mathcal{F}|$, where Markowitz *et al.* (2021) assemble one large KKT system over all variables.

To decide when a blocked asset should re-enter, the code forms the reduced gradients

$$\gamma = \Sigma\, \alpha + C^{\mathsf{T}} \nu^{\alpha}, \qquad \delta = \Sigma\, \beta + C^{\mathsf{T}} \nu^{\beta} - \mu, \tag{7}$$

where ν stacks the equality and active-row multipliers, so that $\gamma_i + \lambda \delta_i$ is (proportional to) the bound multiplier of a blocked asset i along the segment; the products $\Sigma\alpha$, $\Sigma\beta$ go through the covariance operator of §5.

4.2. Event detection

The segment stays optimal until one of four events changes the active set. Two act on variables through the box bounds:

- **Free → blocked.** A free weight $w_i = \alpha_i + \lambda \beta_i$ meets its upper bound at $\lambda_i = (u_i - \alpha_i)/\beta_i$ when $\beta_i < 0$, or its lower bound at $\lambda_i = (\ell_i - \alpha_i)/\beta_i$ when $\beta_i > 0$.

- **Blocked → free.** A blocked asset's multiplier changes sign at $\lambda_i = -\gamma_i/\delta_i$, admissible at the upper bound with $\delta_i < 0$ or the lower bound with $\delta_i > 0$.

The other two are the same moves on the rows of $Gw \le h$, described in §4.3. The candidates form a matrix $L \in \mathbb{R}^{(n+p)\times 4}$, one row per variable or inequality row, with $-\infty$ where an event cannot occur, under two numerical guards:

- **Slope floor.** A slope at round-off level is treated as zero. The floor is $\sqrt{\varepsilon_{\text{mach}}}$ in the problem's own units, so the trace is invariant to rescaling μ or Σ (Appendix A.1).
- λ **window.** Candidates above the current λ, beyond a relative 10^{-10}, lie outside the segment and are discarded; the same window decides which events are tied.

The next turning point is $\lambda^\star = \max_{i,t} L_{it}$; if $\lambda^\star \le 0$ the frontier is complete.

4.3. General linear inequalities

A row of $Gw \le h$ is the analogue of a box bound: either *active* ($g_j^\mathsf{T} w = h_j$) or slack. While active it is carried through the reduced solve as an extra equality row, stacked under A into $C_\mathcal{F} = [\, A_\mathcal{F}; G_{\mathcal{S}\mathcal{F}} \,]$, so each step is still one solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ (through the operator interface) feeding an $(m + |\mathcal{S}|)$-dimensional Schur complement. The box bounds stay a separate per-variable active set, so the box-only problem pays nothing for the generalisation.

The stacked solve (5) returns an affine multiplier $\eta_j(\lambda) = \eta_j^\alpha + \lambda\, \eta_j^\beta$ for each active row, and the two row events mirror the box events:

- **Activate.** An inactive row with slack $s_j(\lambda) = (g_j^\mathsf{T}\alpha - h_j) + \lambda\, g_j^\mathsf{T}\beta$ binds at $\lambda_j = (h_j - g_j^\mathsf{T}\alpha)/(g_j^\mathsf{T}\beta)$, admissible when $g_j^\mathsf{T}\beta < 0$ (the slack rises as λ falls).
- **Release.** An active row releases at $\lambda_j = -\eta_j^\alpha/\eta_j^\beta$, where its multiplier reaches zero, admissible when $\eta_j^\beta > 0$.

These fill the trailing p rows of L, so a group cap or factor-exposure band is traced along the whole frontier (§7.1). Each active row adds a column to the solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ and a row to the Schur complement; with many active rows that term, not the covariance solve, dominates a step. A row that is tight but a combination of the active rows on the free set is supported: at the first vertex only an independent subset of the tight rows is made active (§3), and during the trace such a row has zero slope, $g_j^\mathsf{T}\beta = 0$ (since $C_\mathcal{F}\beta_\mathcal{F} = 0$), so its activation event does not fire. The active rows themselves must stay linearly independent on the free set, or the Schur complement is singular, as when several caps bind at once in the same direction. Before each solve the rows are tested, and a dependent set is refused with a `DegenerateProblemError` at that λ.

4.4. Gross-exposure caps

A cap $\|w\|_1 \le c$ on gross exposure (under a full budget, $c = 1.3$ is a 130/30 book) is polyhedral but not a single row of G. The code lifts it. Each asset whose box straddles zero is split into a long and a short leg, $w_i = x_i^+ - x_i^-$ with $x_i^+ \in [0, u_i]$ and $x_i^- \in [0, -\ell_i]$; a long-only or short-only asset keeps one leg, so a long-only problem is not enlarged. With $w = Px$ for the

signed leg-to-asset map, the cap becomes the one row $\mathbf{1}^\mathsf{T} x \le c$ appended to GP, the other constraints carry over as AP and GP, and the mean becomes $P^\mathsf{T}\mu$. The lifted program is an instance of (1) with the same λ, traced by the loop above and mapped back through P (validated in §7.1).

The lifted covariance $P^\mathsf{T}\Sigma P$ is singular, since raising both legs of one asset leaves w unchanged. The loop never meets that singularity: both legs of an asset are never free together, so every free block is a signed principal submatrix of Σ, served through the operator interface without forming $P^\mathsf{T}\Sigma P$, and a factor backend keeps its Woodbury solve. Appendix A.2 gives the argument and two edge cases of the first vertex.

4.5. Anti-cycling and termination

On degenerate problems (tied returns, exchangeable assets, several constraints active at once) several events can share $\lambda^\star$, and a naive rule can cycle. Among all events within tolerance of $\lambda^\star$ the implementation picks the lowest coordinate index, and within it the lowest event column, and flips that coordinate alone. Call a partition (free set, blocked bounds, active rows $\mathcal{S}$) *valid* at λ when its segment respects the bounds and the inactive rows there and its multipliers have the right signs.

Proposition 1. *Let $\Sigma \succ 0$, and suppose that at every turning point the constraints holding with equality (the rows of A, the rows of G with $g_j^\mathsf{T} w = h_j$, and the bounds on which a weight sits) have linearly independent gradients. Then, in exact arithmetic, the walk never returns to a partition it has left, and it reaches $\lambda = 0$ after finitely many turning points.*

Outline. A partition stays valid while finitely many conditions, each affine in λ, hold; once one fails below a turning point it fails for every smaller λ, so a partition never recurs at a later λ, and without ties every step uses a new one. At a tie, the slope of the next segment solves a strictly convex QP whose KKT system is a linear complementarity problem with a positive-definite matrix, and the implementation's lowest-index single flip is Murty's least-index principal pivot on it, which is finite. Appendix D gives the full argument, including the correspondence between the event types of the code and the complementary pairs of the problem. □

The proposition holds in exact arithmetic and under its hypotheses; what it adds is that the walk never revisits a partition. Two weaker statements hold without them. Because $\Sigma \succ 0$ the optimiser $w(\lambda)$ is unique, and for a fixed set of active constraints it is affine in λ on the interval where that set is optimal; there are finitely many such sets, so the frontier itself has finitely many turning points whether or not the constraints are independent. And an iteration cap of $100(n+p+1)$ stops every trace: a trace that reaches it raises an error and never returns a partial frontier, and no trace in Appendix B used more than 1.5% of it. In exact arithmetic, then, the one thing left unproved outside the hypotheses is that the walk leaves every tie without revisiting a partition. This concerns a degenerate maximum-return vertex, resolved from a vertex of its optimal dual set, or, under tied maximum returns, from the minimiser of the quadratic term on the tied face (§3); a singular Σ that the active rows keep positive definite on their null space, outside $\Sigma \succ 0$ (Appendix B); the lifted problem of §4.4, whose singular covariance masks the competing leg events; and ties with dependent constraints, where the matrix M of Appendix D is only positive semidefinite and the least-index result

no longer applies (a partition with dependent active rows is refused, §4.3). Appendix B lists which test families satisfy the hypotheses. In floating point the tie window and the slope floor make the computed signs agree with the exact ones, which is not proved either. At run time every turning point is validated against the bounds, $Aw = b$ and $Gw \leq h$ before it is accepted, and the KKT certificate of §7.1 checks the traces of §7 and Appendix B for optimality.

When no admissible event has a positive ratio the walk has reached the minimum-variance portfolio and appends a final turning point $w = \alpha$ at $\lambda = 0$.

Algorithm 1. Critical Line Algorithm (as implemented in **cvxcla**).

Input: μ, Σ, ℓ, u, A, b, G, h, tolerance τ.

1. Compute the maximum-return vertex (§3); record its free set and active rows $\mathcal{S}$ with $\lambda \leftarrow \infty$.
2. **While** $\lambda > 0$:
   (a) Fix $w_{\mathcal{B}}$ at the active bounds and stack $C_{\mathcal{F}} = [\, A_{\mathcal{F}}; G_{\mathcal{SF}} \,]$.
   (b) Solve against $\Sigma_{\mathcal{FF}}$ for Y, z^{α}, z^{β}; solve the Schur system (5); back-substitute (6) for α, β; form γ, δ (7).
   (c) Assemble L (§4.2, §4.3) under the slope floor and λ window; $\lambda^{\star} \leftarrow \max L$; **if** $\lambda^{\star} \leq 0$ **break**.
   (d) Pick the event by the least-index rule (§4.5), flip that coordinate's status, set $\lambda \leftarrow \lambda^{\star}$, append $w = \alpha + \lambda^{\star}\beta$.
3. Append $w = \alpha$ at $\lambda = 0$ and **return** the turning points.

5. The covariance operator abstraction

The design separates the path-following algorithm from the storage and solution of the quadratic form (Figure 2). The turning-point loop needs only three operations on the covariance: a matrix–vector product Σx, a multi-column solve against the free block $\Sigma_{\mathcal{FF}}^{-1} R$, and the free-to-blocked cross product $\Sigma_{\mathcal{FB}}\, x_{\mathcal{B}}$, which is formed from the first. A backend supplies them through the protocol `QuadraticForm` (alias `CovarianceOperator`, the `SymmetricOperator` contract of **cvx-linalg**), whose five members are listed in Table 2; two of them serve the gross-exposure lift and the singularity guard. The algorithm never assumes an explicit matrix, and it assumes of a backend only that Σ is symmetric and that every free block it solves is positive definite on the null space of the active rows, which `rcond_free` lets it check before trusting the solve (Appendix B). Four backends ship with the package, and a user can add others without touching the loop (§5.2).

Dense. `DenseCovariance` wraps an explicit matrix and solves the free block by Cholesky (`scipy.linalg.cho_factor`), falling back to LU on the rare block that is not numerically positive definite. It reproduces the classical CLA and is the reference backend.

Incremental dense. `IncrementalDenseCovariance` traces the same turning points but *maintains* $\Sigma_{\mathcal{FF}}^{-1}$: because the free set changes by one asset per turning point, a bordered

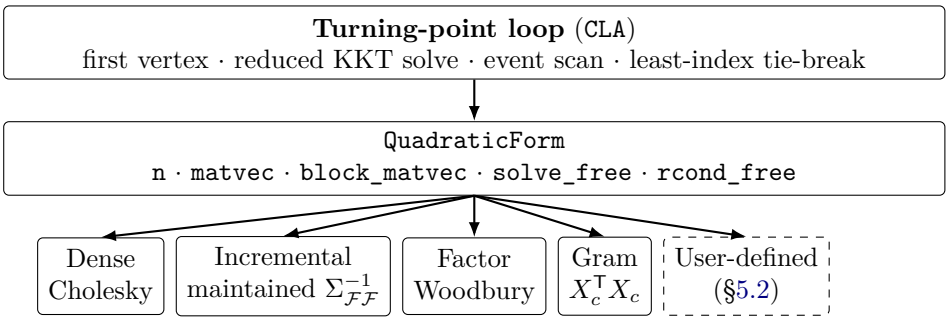


Figure 2: The covariance enters the turning-point loop only through the `QuadraticForm` protocol; the backends below it are interchangeable.

Member	Returns	Used for
`n`	the number of assets	sizes
`matvec(x)`	Σx, for a vector or a matrix x	gradients, the cross product
`block_matvec(rows, cols, v)`	$\Sigma_{\text{rows,cols}}\, v$	the gross-exposure lift (§4.4)
`solve_free(free, R)`	$\Sigma_{\mathcal{FF}}^{-1} R$	the reduced KKT solve (§4.1)
`rcond_free(free)`	the reciprocal condition number of $\Sigma_{\mathcal{FF}}$	the singularity guard

Table 2: The `QuadraticForm` protocol. Index sets are integer arrays into $\{0, \dots, n-1\}$.

rank-one update or a symmetric deletion costs $O(n_{\mathcal{F}}^2)$ against a fresh $O(n_{\mathcal{F}}^3)$ factorisation, which makes a trace about twice as fast at $n = 320$, three times at $n = 640$, and seven times at $n = 2560$ (§7.3). A non-positive Schur pivot or a multi-asset change triggers a full refactorisation. The backend passes the same KKT certificate as the dense one (Table 4), but a maintained inverse carries round-off from step to step over the roughly n updates of a trace, so it is opt-in.

Factor (diagonal-plus-low-rank). `FactorCovariance` represents

$$\Sigma = \operatorname{diag}(d) + U\,\Delta\,U^{\mathsf{T}}, \qquad d \in \mathbb{R}^n_{>0},\ U \in \mathbb{R}^{n\times K},\ \Delta \in \mathbb{R}^{K\times K}, \tag{8}$$

the form of factor models (Sharpe 1963) and of eigenvalue-clipped covariances. The free-block solve never forms $\Sigma_{\mathcal{FF}}$; it uses the Woodbury identity

$$\Sigma_{\mathcal{FF}}^{-1} = D_{\mathcal{F}}^{-1} - D_{\mathcal{F}}^{-1} U_{\mathcal{F}}\, W^{-1}\, U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1}, \qquad W = \Delta^{-1} + U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1} U_{\mathcal{F}} \in \mathbb{R}^{K\times K}, \tag{9}$$

so a solve costs $O(n_{\mathcal{F}}K^2 + K^3)$ rather than $O(n_{\mathcal{F}}^3)$, storage is $O(nK)$, and the cross product is $U_{\mathcal{F}}\,\Delta\,(U_{\mathcal{B}}^{\mathsf{T}} x_{\mathcal{B}})$. The same free-block algebra serves a single constrained solve as well: Schmelzer and Stoll (2026) reach the free block of a non-negative QP through the same Woodbury inverse and factored products.

The admissible model is $d > 0$ and Δ symmetric positive semidefinite. `FactorCovariance` folds Δ into the loadings when it is built: with $\Delta = V\Lambda V^{\mathsf{T}}$ it stores $U' = UV_+\Lambda_+^{1/2}$, the directions of positive variance, and an identity inner matrix, which is the same Σ. The Woodbury solve (9) then runs with U' and $W = I + U_{\mathcal{F}}'^{\mathsf{T}} D_{\mathcal{F}}^{-1} U_{\mathcal{F}}'$ and never inverts Δ. A singular

factor covariance, from a factor with zero variance or perfectly correlated factors, is therefore admissible, and its rank K is the number of directions kept. An indefinite Δ is refused: a signed low-rank term can still give a positive-definite Σ, but W would then be indefinite, so such a covariance is passed as a dense matrix. Since $d > 0$, W is positive definite, and the factor solve needs no guard of its own beyond the one on $\Sigma_{\mathcal{F}\mathcal{F}}$. Against the dense trace of the same Σ, the factor trace has the same turning points and weights equal to 2.8×10^{-12}, with KKT residuals below 2.3×10^{-13}, for Δ with condition numbers up to 10^{14}, loadings collinear to within 10^{-10}, and the full rank $K = n = 120$ (`make_figures.py validate-factor`). Every active constraint row adds a column to the solve and a row to the Schur complement, so with many active rows that term dominates the cost. A low-rank *approximation* of a sample covariance traces the frontier of the approximation, which is a different frontier (§7.7).

Data matrix (Gram). A sample covariance is a Gram matrix: for the column-centred returns $X_c \in \mathbb{R}^{T \times n}$, $\Sigma = X_c^{\mathsf{T}} X_c/(T-1)$. `GramCovariance` implements the protocol straight from X_c in $O(Tn)$ storage. When $T < n$ the matrix is singular, and once the free set outgrows the data rank the free block is singular too; the active rows still hold it on their null space, where it is solved (Appendix B). An optional ridge θI (the argument `ridge`) makes Σ positive definite, a different, regularised frontier, and the solve is then Woodbury in the T-dimensional observation space. The saving is in storage: when the dense matrix fits, `DenseCovariance` is faster per step, and the structured route pays only while T (or K) stays well below n (§7.4).

5.1. Regularization through the operator

A quadratic penalty $\frac{\theta}{2} w^{\mathsf{T}} M w$, with $\theta \geq 0$ and M symmetric positive semidefinite and independent of λ, changes only the Hessian, so the path stays piecewise linear and the penalty is the substitution $\Sigma \mapsto \Sigma + \theta M$: a ridge ($M = I$) or a factor-exposure penalty ($M = BB^{\mathsf{T}}$ for exposures B). `CLA` has no penalty argument; the regularized form is supplied as the covariance, a bundled or a user-written `QuadraticForm` (§5.2). `GramCovariance` with $\theta > 0$ and `FactorCovariance` already are such forms. A rank-deficient sample covariance needs neither: its frontier is traced as it is, as long as the active rows keep the free block positive definite on their null space (Appendix B). A small ridge or a factor model is the fix when they do not, and a choice of estimator otherwise (§7.7). A penalty centred on a nonzero portfolio w_0 (tracking error, quadratic turnover) adds the linear term $-\theta\, w^{\mathsf{T}} M w_0$, which the interface does not expose; nonquadratic penalties such as linear transaction costs lie outside the method.

5.2. Writing a backend

A backend is a subclass of `QuadraticForm` that implements the five members of Table 2. For a block-diagonal covariance, assets in groups with no cross-group covariance (a structure the package does not ship), the free-block solve splits into one Cholesky solve per block. The two members that carry the structure are below; `n`, `matvec` and `block_matvec` are a few lines each, and the complete, runnable class is in the package documentation (`docs/custom-backend.md`).

```
class BlockDiagonal(QuadraticForm):   # Sigma = blockdiag(S_1, ..., S_m)
    def solve_free(self, free, rhs):
        out = np.empty_like(rhs, dtype=float)
        # one (positions, free sub-block) pair per group
```

```
        for pos, sub in self._parts(free):
            out[pos] = cho_solve(cho_factor(sub), rhs[pos])
        return out

    def rcond_free(self, free):
        eig = np.concatenate(
            [np.linalg.eigvalsh(sub) for _, sub in self._parts(free)]
        )
        return eig.min() / eig.max()
```

`CLA` runs on it unchanged: on four groups of five assets it traces the same 19 turning points as the dense matrix. The package's test suite, `tests/test_custom_backend.py`, checks the same with general constraints and with a gross-exposure cap, which goes through `block_matvec`. The frontier is exact to the accuracy of `solve_free`; an approximate solver traces the frontier of the matrix it effectively applies, and the KKT certificate of §7.1 checks the result. A sparse matrix with a sparse Cholesky factor, a Kronecker product, or a matrix-free operator with an iterative `solve_free` fit the same five members.

6. Using the package

The package (Schmelzer 2026c) is installed with `pip install cvxcla`; it is pure Python and needs **NumPy**, **SciPy** and **cvx-linalg** (Computational details). The workflow goes through a single class, `CLA`. The user supplies μ, the covariance (a dense array or any backend of §5), the bounds, and the constraints; constructing the object traces the whole frontier. The example traces a small long-only, fully-invested problem and reads off the maximum-Sharpe portfolio.

```
import numpy as np
from cvxcla import CLA, FactorCovariance

rng = np.random.default_rng(1)
n, k = 6, 2
d = rng.uniform(0.1, 0.5, n)            # idiosyncratic variances
U = rng.standard_normal((n, k))         # factor loadings
Delta = np.diag(rng.uniform(0.5, 2.0, k))
Sigma = np.diag(d) + U @ Delta @ U.T    # a dense 2-factor covariance
mean = rng.standard_normal(n)

cla = CLA(
    mean=mean, covariance=Sigma,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1),    # fully invested: sum(w) = 1
)
print("turning points:", len(cla.turning_points))
sr, w = cla.frontier.max_sharpe
print("max Sharpe:", round(sr, 4))
print("weights:", np.round(w, 3))
```

```
turning points: 7
max Sharpe: 2.4037
weights: [0.    0.    0.704 0.296 0.    0.   ]
```

Switching to a structured backend changes only the covariance argument; because the factor backend represents the *same* Σ, it returns the same frontier to round-off.

```
factor = FactorCovariance(d=d, u=U, delta=np.diag(Delta))
cla_factor = CLA(
    mean=mean, covariance=factor,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1),
)
w_dense = np.array([t.weights for t in cla.turning_points])
w_factor = np.array([t.weights for t in cla_factor.turning_points])
print("identical frontier:", np.allclose(w_dense, w_factor))
```

```
identical frontier: True
```

A fluent builder, `CLA.problem`, assembles the same arguments step by step through methods such as `.budget`, `.equality` and `.inequality`; an extra `.equality` row, such as a fixed sector weight, makes the first turning point the linear program of §3.

6.1. The interface

`CLA` takes the problem as keyword arguments, all **NumPy** arrays of floats, and traces the frontier in its constructor. Required are `mean` (μ, length n), `covariance` (an $n \times n$ array, which is wrapped as a `DenseCovariance`, or any backend of §5), `lower_bounds` and `upper_bounds` (length n), and `a`, `b` ($m \times n$ and length m; a budget is `a = np.ones((1, n))`, `b = np.ones(1)`). Optional are `g`, `h` ($p \times n$ and length p) for $Gw \leq h$, `leverage` for the cap $\|w\|_1 \leq c$, and `tol` (default 10^{-5}), the tolerance for deciding that a weight sits on a bound and for validating each turning point; the event ordering uses its own relative window of at most 10^{-10} (§4.2). The arrays (and a dense covariance) are copied into read-only arrays when the object is built, so changing the caller's arrays afterwards cannot put the stored turning points out of step with the data; a backend is used as given. Non-finite data are refused at this point, as are NaN bounds. Redundant equality rows are dropped, and `cla.a`, `cla.b` then hold the reduced system.

The result is `cla.turning_points`, a list of `TurningPoint` objects with the `weights`, the value `lamb` of λ, the boolean mask `free` of free assets, and the mask `active_ineq` of active rows of G. The maximum-return vertex appears twice, at $\lambda = \infty$ and again at the first finite turning point, where the path leaves it; `len(cla)` is the number of turning points. `cla.frontier` wraps them as a `Frontier` (`types.py`), which exposes the `weights`, `returns`, `variance`, `volatility` and `sharpe_ratio` of the turning points and a `plot` method. Because the frontier is piecewise linear in weight space, `interpolate(num)` returns intermediate portfolios with no discretisation error.

6.2. The frontier and the maximum-Sharpe portfolio

`max_sharpe` maximises $\mu^{\mathsf{T}} w / \sqrt{w^{\mathsf{T}} \Sigma w}$, the Sharpe ratio with a zero risk-free rate (supply excess returns as μ otherwise), over the computed frontier. On a segment $w(t) = (1-t)\, w_0 +$

$t\,w_1$, $t \in [0,1]$, the return is affine and the variance quadratic in t,

$$S(t) = \frac{a_0 + a_1 t}{\sqrt{c_0 + c_1 t + c_2 t^2}}, \qquad \begin{aligned} a_0 &= \mu^{\mathsf{T}} w_0, & c_0 &= w_0^{\mathsf{T}} \Sigma w_0, & c_2 &= \Delta^{\mathsf{T}} \Sigma \Delta, \\ a_1 &= \mu^{\mathsf{T}} \Delta, & c_1 &= 2\, w_0^{\mathsf{T}} \Sigma \Delta, & \Delta &= w_1 - w_0, \end{aligned} \tag{10}$$

and the numerator of $S'(t)$ is affine in t (its t^2 terms cancel), so S has at most one stationary point,

$$t^\star = \frac{a_0 c_1 - 2 a_1 c_0}{a_1 c_1 - 2 a_0 c_2}. \tag{11}$$

The maximum on the segment is the largest of $S(0)$, $S(1)$ and, when $0 < t^\star < 1$, $S(t^\star)$; a stationary point outside the segment is ignored, an optimum at a turning point is found as an end point, and when $a_1 c_1 = 2 a_0 c_2$ there is no stationary point and only the end points are compared. The code applies this rule to every segment of the frontier and returns the best candidate, so the result is the exact maximum over the computed frontier, without relying on the ratio being unimodal, at the cost of one matrix–vector product with Σ per segment.

Points of zero variance are skipped (`sharpe_ratio` reports NaN there); with no positive variance `max_sharpe` raises a `ValueError`, and with no positive expected return it warns and returns the least negative ratio. Whether the ratio, computed from estimated μ and Σ, predicts future performance is a statistical question outside the package (§7.7).

6.3. Supported inputs and errors

Table 3 sorts the inputs into kinds, each with its own exception and remedy, and each row is a test in the package (`tests/test_errors.py` and `tests/test_degenerate_vertex.py`). All are raised by the constructor. The package errors derive from `CLAError` and also from the built-in exception raised before (`ValueError`, or `RuntimeError`), so existing handlers keep working.

A sample covariance from fewer observations than assets is a case that used to be declined and no longer is. It is singular, and its free block becomes singular partway along the frontier. The budget row still holds the free assets on its null space, where the block stays positive definite, so the trace goes on there (Appendix B). A ridged Gram backend (§5) traces a different, regularised frontier:

```
from cvxcla import GramCovariance

returns = 0.01 * rng.standard_normal((4, n)) + 0.001    # 4 days, 6 assets:
    rank 3
long_only = dict(lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
                 a=np.ones((1, n)), b=np.ones(1))
cla = CLA(mean=returns.mean(axis=0),
          covariance=np.cov(returns, rowvar=False), **long_only)
ridged = CLA(mean=returns.mean(axis=0),
             covariance=GramCovariance(returns, ridge=1e-4), **long_only)
print("turning points:", len(cla), "| with a ridge:", len(ridged))
```

```
turning points: 5 | with a ridge: 7
```

Release 2.3.3 declined the first of these. Every turning point of the trace above matches an interior-point solve of the same quadratic program.

Input	Remedy
Malformed input: `ValueError`, before the walk	
Non-symmetric Σ; non-finite data or NaN bounds; an indefinite factor Δ; wrong shapes	correct the data
Infeasible: `InfeasibleProblemError`, before the walk	
Crossed bounds; an unreachable budget; constraints with no feasible point; dependent equality rows that contradict the others	change the constraints
Harmless degeneracy: traced, nothing raised	
Redundant equality rows; duplicated, never-binding or locally redundant inequality rows; a degenerate maximum-return vertex whose optimal dual set has a vertex, and tied maximum returns (§3); a singular covariance whose free blocks the active rows keep positive definite (Appendix B)	–
Unsupported degeneracy: `DegenerateProblemError`	
Before the walk: a degenerate vertex whose optimal dual set has no vertex, as when a tight row touches only assets fixed at both bounds; infinite bounds that make the return unbounded	perturb the bounds or constraints
During the walk: a free block singular even on the null space of the active rows, or active rows dependent on the free set	a ridge or factor model; loosen a cap
Numerical failure: `NumericalError`, during the walk	
Iteration cap; projection fails; a turning point violates a constraint	regularise Σ or relax `tol`; report it

Table 3: Supported and unsupported inputs, grouped by the exception each raises and when, with the usual remedy. "Before the walk" is input validation and the first vertex; "during the walk" is a turning point of the trace.

7. Numerical experiment

The experiments answer three questions, in this order: whether the trace is correct (§7.1); whether its speed on large problems comes from the structured solve rather than from bookkeeping (§7.2–§7.4); and how it compares with the alternatives, a QP swept over a grid and a parametric QP solver (§7.5, §7.6). A real S&P 500 frontier closes the section (§7.7); further checks are in Appendices A and B. Timings are the median of three repetitions (bands span their range), each method and size in a freshly spawned interpreter, on the machine of Computational details. Two complete runs of the scaling sweep agree to within 10% (median 3%) for traces of 10 ms or more and to within 40% below that, so ratios are rounded accordingly and those at small n are indicative only.

7.1. Correctness: a reference solver and a KKT certificate

On 40 real assets (§7.7; condition number ≈ 200) the CLA traces 29 turning points. At the midpoint λ of each of the 27 interior segments we re-solve (1) with **OSQP** (Stellato, Banjac, Goulart, Bemporad, and Boyd 2020), polished and at tight tolerances, and compare

Problem	cond(Σ)	points	primal	station.	dual	compl.
S&P 500, 40 assets	2.1×10^{2}	83	2×10^{-16}	4×10^{-17}	0	4×10^{-22}
S&P 500, 494 assets	1.1×10^{5}	284	3×10^{-16}	2×10^{-14}	0	5×10^{-27}
incremental backend	1.1×10^{5}	284	7×10^{-16}	2×10^{-14}	0	3×10^{-23}
last 600 days	4.7×10^{5}	335	4×10^{-16}	2×10^{-13}	0	3×10^{-23}
Factor market, 320 assets	1.3×10^{3}	965	3×10^{-14}	9×10^{-14}	0	0
30 assets:						
neutrality row	8.3×10^{1}	98	5×10^{-14}	1×10^{-15}	0	2×10^{-15}
sector caps	8.3×10^{1}	89	4×10^{-15}	2×10^{-15}	0	1×10^{-15}
130/30 cap	8.3×10^{1}	119	4×10^{-15}	9×10^{-16}	3×10^{-16}	4×10^{-15}

Table 4: Largest KKT residual of each condition (primal feasibility, stationarity, dual feasibility, complementarity) over all points tested on a trace. Primal feasibility is absolute; the other three are relative to $\max(1, \|\Sigma w\|_\infty, \lambda\|\mu\|_\infty)$. The factor market is traced with the factor backend and checked against its dense Σ; the 130/30 problem is checked in its lifted long/short form. Produced by `make_figures.py validate-kkt`.

with the interpolated CLA weights: the median discrepancy is 2×10^{-15} and the maximum 6×10^{-8}, solver precision (`make_figures.py validate-exact`). Since each segment is affine, one interior point certifies it. The step `make_figures.py validate-constraints` validates two constrained frontiers the same way on a 30-asset factor market: a budget plus a neutrality row (the LP first vertex) and a budget with per-sector caps (an inequality active at 30 of 31 turning points). Both match to 6×10^{-14} or better. A third frontier is a 130/30 book: the budget, shorts down to -30% per asset, and the gross-exposure cap $\|w\|_1 \le 1.3$ of §4.4, which binds at 32 of 49 turning points; it matches to 1.4×10^{-12}. The reference QP models the cap with auxiliary variables $t \ge |w|$, independently of the lift.

A reference solver only shows that two codes agree; a KKT certificate tests the trace itself (`make_figures.py validate-kkt`). At three interior points of every segment, and at two values of λ above the first turning point, the active set is read off the interpolated weights (within 10^{-9} of a bound or of h), the multipliers are solved from stationarity on the free coordinates, and primal feasibility, stationarity, dual feasibility and complementary slackness are each measured. Neither the multipliers nor the partition come from **cvxcla**, and the certificate always uses an explicit dense Σ, never the backend that traced the frontier. Where the free coordinates do not determine the multipliers, at a vertex with no free weight, a bounded least-squares fit asks whether sign-feasible multipliers exist. Primal feasibility is absolute, the other conditions relative to $\max(1, \|\Sigma w\|_\infty, \lambda\|\mu\|_\infty)$. Over 2257 points on eight traces (Table 4), including the incremental and factor backends, the general constraints and the lifted 130/30 problem, no residual exceeds 2×10^{-13}. The test does detect errors: weights shifted by 10^{-3} give a residual of 2.5×10^{-7}, and weights paired with the λ of another segment 3.3×10^{-4}.

7.2. Runtime versus problem size

We trace the full frontier for $n \in \{20, 40, \ldots, 2560\}$ assets, each from a ground-truth $K = 10$ factor model, so the dense and factor backends run the same loop on the same Σ and differ only in the free-block solve (`make_figures.py scaling`). The free set is not small: on this market it grows to all n assets, with a median of $n/2$ over the trace. With the roughly n steps observed here, a dense trace is about n solves of $O(n_{\mathcal{F}}^3)$ each, $O(n^4)$ operations, while

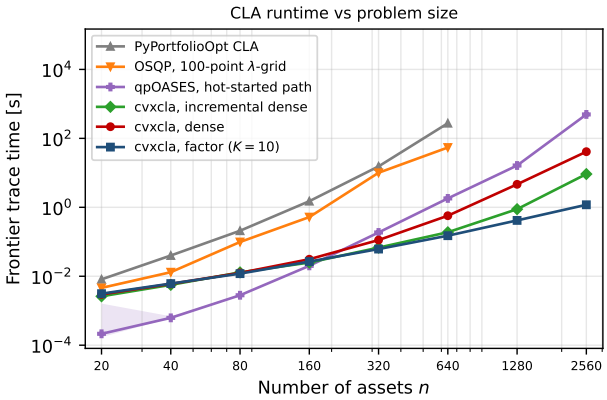


Figure 3: Frontier trace time against n (log–log). Exponents p of time $\approx n^p$, fitted over the last four sizes each method was timed at with no overhead removed: 2.9 dense, 2.4 incremental dense and 1.4 factor (n = 320–2560), 3.4 **PyPortfolioOpt** and 3.2 **OSQP** (80–640), 3.7 **qpOASES** (320–2560); they describe these sizes on this machine, not asymptotic complexity. **PyPortfolioOpt** and a warm-started **OSQP** grid of 100 values of λ (§7.5) are timed up to $n = 640$, and **qpOASES** hot-started along the whole path (§7.6) up to $n = 2560$. The methods deliver different outputs: **cvxcla** and **PyPortfolioOpt** return every turning point, the **OSQP** grid 100 portfolios, and **qpOASES** only the minimum-variance endpoint, so its time covers the path following alone.

the per-step cost of the factor backend is $O(n_{\mathcal{F}}K^2 + nK)$, so $O(n^2K)$ in total. At small n a step's fixed overhead hides both rates, and the two backends are at parity up to $n \approx 80$. Over the largest four sizes (320 to 2560) the fitted exponents are ≈ 2.9 for the dense backend and ≈ 1.4 for the factor backend (Figure 3), below the operation counts because per-step overhead has not yet given way entirely to arithmetic. The factor backend is about 4× faster at $n = 640$, 11× at 1280 and 35× at $n = 2560$, where it traces the 2561 turning points in about 1.2 s against 41 s (Figure 3).

The factor backend also never holds an $n \times n$ matrix. At $n = 2560$ its covariance takes 0.22 MiB against 50 MiB, and the trace's working memory, the `tracemalloc` peak less what the finished trace retains, is 0.84 MiB against 100 MiB (Python objects and NumPy buffers; BLAS workspace is not counted). The stored frontier, n weights per turning point, is $O(n^2)$, 50 MiB at $n = 2560$, for either backend.

For external context we time the `CLA` of **PyPortfolioOpt** (Martin 2021), the most familiar open implementation, based on Bailey and López de Prado (2013). It scales like $\approx n^{3.4}$ and takes about 275 s at $n = 640$, against 0.6 s for **cvxcla**'s dense backend; most of that gap is how that code organises its per-step work (Appendix C), which §7.3 separates from the gain of the structured solve. The two agree at the minimum-variance endpoint but not along the whole frontier: two bugs make **PyPortfolioOpt** drop genuine turning points (Appendix C).

7.3. Where the speed comes from

A fairer comparison sets **cvxcla**'s own dense backend against its opt-in incremental one, `IncrementalDenseCovariance` (§5), which runs the same loop but maintains $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ by rank-

one updates and reports the same turning points. A trace is then $O(n^3)$, the measured exponent ≈ 2.4, and the incremental backend is about $3\times$ faster than the dense one at $n = 640$ and $4.5\times$ at $n = 2560$: block elimination is not the fastest dense strategy. The factor backend is still about $1.3\times$ faster than the incremental one at $n = 640$ and $8\times$ at $n = 2560$, so the speed comes from the structured solve. Updating the Woodbury capacitance W incrementally would add little at $K = 10$, where the $O(n)$ work of each step (event scan, products with Σ, bookkeeping) dominates, and matters only at large K, where the dense backend is already faster (§7.4).

7.4. Runtime versus factor rank

The factor backend's advantage must fade as K approaches n. At fixed $n = 320$ and $K \in \{20, \ldots, 320\}$ (`make_figures.py rank-scaling`), the dense time is flat in K while Woodbury grows as $O(nK^2 + K^3)$: the factor backend is about $1.8\times$ faster at $K = 20$, at parity near $K = 160$, and $\approx 1.7\times$ slower at $K = 320$. It pays off in the regime real risk models inhabit, a few tens of factors over hundreds or thousands of assets.

7.5. A general-solver baseline: a QP swept over λ

General toolkits (**Riskfolio-Lib** (Cajas 2021), **skfolio** (Nicolini, Manzi, and Delatte 2025), **fPortfolio** (Würtz, Chalabi, Chen, and Ellis 2009), **CVXR** (Fu, Narasimhan, and Boyd 2020)) hand a single risk target to a general solver, so a frontier means re-solving (1) on a grid. We make the grid as favourable as we can: **OSQP** (Stellato *et al.* 2020) is set up once, each λ changes only the linear term, and each solve warm-starts from the last, over a fixed grid of 100 values (`make_figures.py scaling`). The sweep grows like $\approx n^{3.2}$ and is slower at every size, from about $1.5\times$ the dense trace at $n = 20$ to about $95\times$ at $n = 640$ (54 s against 0.6 s). On the real frontier of §7.7 it is about $240\times$ slower at tolerance 10^{-6}, and at 10^{-5} it misses points by 10^{-3} (`make_figures.py osqp`). For a single risk target a general solver is the natural choice; for the whole frontier on a structured covariance, the trace is what separates **cvxcla** from these toolkits.

7.6. A parametric QP solver

qpOASES (Ferreau *et al.* 2014) solves parametric QPs by an online active-set strategy: a hot start from one QP to the next follows the homotopy between them and changes the working set at every breakpoint on the way. Hot-started from the maximum-return end to $\lambda = 0$, it walks the same path as the CLA. We run it through **CasADi** (Andersson, Gillis, Horn, Rawlings, and Diehl 2019), solving once above the first finite turning point and hot-starting once to $\lambda = 0$ (`make_figures.py scaling`).

The two codes follow the same path: **qpOASES** makes exactly as many working-set changes as **cvxcla** reports turning points $(21, 42, \ldots, 2561)$, and its endpoint matches the minimum-variance portfolio to $\sim 2 \times 10^{-16}$. On small problems it is faster, by a factor of several up to $n = 80$, where **cvxcla**'s Python per-step overhead dominates; the two meet near $n = 200$. Beyond that **qpOASES** grows like $\approx n^{3.7}$ and takes about 1.8 s at $n = 640$ and 490 s at $n = 2560$: about $3\times$ and $12\times$ the dense backend, $12\times$ and $420\times$ the factor backend. It also returns only the endpoint, since the turning points it passes are not exposed through the CasADi interface.

7.7. Empirical example: the S&P 500 frontier

The experiment runs off a frozen snapshot shipped with the package: $N = 494$ assets over $T = 1213$ trading days, 2021-07-30 to 2026-05-29, stored as `sp500_pct_returns.parquet` (SHA-256 `b5faa522...9937e`) and produced once by `fetch_sp500.py`, retained for provenance only. That script takes the index constituents listed on the day it was run, downloads their split- and dividend-adjusted closes from **yfinance**, drops names missing more than 5% of the trading days, and forward-fills short gaps. The universe is the index at the end of the window, with survivorship bias, so we use it only as a realistic covariance; it says nothing about investment performance. From the full-history sample moments (condition number $\approx 1.1 \times 10^5$) the CLA traces the long-only frontier, 96 turning points, in about 23 ms; a rank-20 factor approximation traces a near-identical frontier (89 points) in about 15 ms. The frontier spans annualised returns of 0.14–0.89 over volatilities of 0.10–0.87, with an in-sample maximum Sharpe ratio near 2.7 (`make_figures.py real-frontier`).

Covariance estimators as backends. Because **cvxcla** traces the frontier of whatever covariance it is given, a shrinkage or factor estimate is simply another backend. Most remedies for a noisy sample covariance (Johnstone 2001) return the diagonal-plus-low-rank form (8): approximate factor models (Chamberlain and Rothschild 1983), POET (Fan, Liao, and Mincheva 2013), eigenvalue clipping (Laloux, Cizeau, Bouchaud, and Potters 1999), or Ledoit–Wolf shrinkage (Ledoit and Wolf 2004), $(1-\rho)S+\rho F$. On the same universe, changing only the covariance argument, **cvxcla** traces the frontier of the sample covariance (96 turning points), the Ledoit–Wolf estimate (94) and a rank-20 PCA model (89); the three maximum-Sharpe portfolios hold 26–29 names (`make_figures.py estimators`). As the window shrinks past $T = n$ the sample covariance loses rank while the shrinkage and factor estimates stay well-conditioned (§5.1). **cvxcla** controls only the computational error; an exact frontier of a noisy estimate is still noisy, and ranking the estimators would take an out-of-sample study.

8. Conclusion

The Critical Line Algorithm computes the whole mean–variance frontier by walking its piecewise-linear path from the maximum-return to the minimum-variance portfolio. **cvxcla** takes each step with one block-eliminated KKT solve, resolves ties by a least-index rule under which the walk provably terminates (Proposition 1), and reaches the covariance only through a five-member operator protocol. A factor risk model therefore yields the same turning points as the dense matrix it represents; with 2560 assets and ten factors it does so about 35 times faster, in under 1 MiB of working memory beyond the stored frontier. The gain is one of scale and structure. Below about 200 assets the per-step overhead of Python dominates, and **qpOASES**, which follows the same path, is faster.

Computational details

The results in this paper were obtained with **cvxcla** 2.6.0 (Schmelzer 2026c). Release 2.7.1 changes its Critical Line Algorithm only in the check that a bound held along a segment is still optimal, for problems with no active rows and with a linear benchmark term, neither of which arises here; the checks of this paper give the same results under it. They

were run with Python 3.14.6 on an Apple M4 Pro (14 cores, 48 GB of memory) under macOS 15.6.1. **NumPy** 2.4.6 and **SciPy** 1.17.1 were linked against Apple's Accelerate BLAS, and the first turning point under a general equality system is found with the HiGHS solver through `scipy.optimize.linprog`. The comparisons use **PyPortfolioOpt** 1.6.0, **OSQP** 1.1.3, and the **qpOASES** bundled with **CasADi** 3.8.1. The estimators of §7.7 use **scikit-learn** 1.9.0, the figures **matplotlib** 3.11.0, and the return data are read with **pandas** 3.0.3 and **pyarrow** 24.0.0.

Software availability. **cvxcla** is released under the MIT license. Release 2.7.1, which ships the experiment script in the form used here, is archived on Zenodo (Schmelzer 2026c) with DOI [**TODO: the Zenodo DOI of this release**]; the source is developed at `https://github.com/cvxgrp/cvxcla`, and every source link in this paper points to the release tag `v2.7.1`. The package declares only lower bounds on its dependencies, and its continuous integration also tests them at those bounds; the experiment script pins the exact versions above. The test suite, some 600 unit and property-based tests (the latter with **Hypothesis**), enforces full line coverage of the library.

Reproducing the results. Every figure, table, and number is produced by one script, `make_figures.py` in the **cvxcla** repository. It declares the versions above as inline (PEP 723) metadata, so with **uv** installed

```
uv run experiments/make_figures.py all
```

installs them into an isolated environment and rebuilds everything. A target name in place of `all` rebuilds one artefact, and `all --quick` skips the two scaling sweeps (§7.2 and §7.4), which take about 40 minutes on the machine above. Random inputs are drawn from fixed seeds and the S&P 500 returns are a committed snapshot (§7.7), so the run needs no network access once the packages are installed. Timings and memory were measured on this machine only and will differ elsewhere; the turning-point counts and accuracy figures should agree up to round-off.

Acknowledgements

The author thanks Stephen Boyd, Trevor Hastie and Philipp Schiele. The first versions of the solver were implemented while the author stayed for a sabbatical at Boyd's group at Stanford University.

A. Implementation details

A.1. The event guards

Slopes are tested against a floor of $\sqrt{\varepsilon_{\text{mach}}}$ in the problem's own units, not against the user tolerance, which would let a tiny but genuine slope go unnoticed. With $m = \|\mu\|_\infty$ and $\sigma = \|\Sigma\mathbf{1}\|_\infty/n$, λ has the scale σ/m. Weight slopes β_i and slack slopes $g_j^\mathsf{T}\beta$ carry units of $1/\lambda$ and are tested against $\sqrt{\varepsilon_{\text{mach}}}\, m/\sigma$; multiplier slopes δ_i, η_j^β carry the units of μ and are

tested against $\sqrt{\varepsilon_{\text{mach}}}\, m$. The λ window is a relative 10^{-10} taken against $\max(|\lambda|, \sigma/m)$, so that it does not vanish as $\lambda \to 0$.

The trace does not depend on the units (`make_figures.py validate-scaling`). Rescaling the returns by c and the covariance by s, each from 10^{-6} to 10^{6}, leaves all 29 turning points of the 40-asset problem in place: the weights agree to 8×10^{-15}, and $\lambda c/s$ to a relative 1.2×10^{-13}. Multiplying the slope floors by any factor from 10^{-6} to 10^{4} leaves the traces of the 40- and 494-asset S&P 500 problems, the 600-day window and the 320-asset factor market bit for bit unchanged. Only at 10^{6} does the floor start to swallow genuine slopes: the 40-asset trace is still unchanged, the 600-day window's weights move by 7×10^{-2}, and the trace of the 494-asset problem and of the factor market is declined. So on these inputs genuine slopes and round-off are about ten orders of magnitude apart. A turning point lost to the floor would leave the later ones feasible but not optimal, which the KKT certificate reports as a stationarity or dual-feasibility residual.

A.2. The gross-exposure lift

Both legs of an asset are never free together on the path. Write g_i for the i-th entry of $\Sigma w - \lambda\mu + A^{\mathsf{T}}\nu + G^{\mathsf{T}}\eta_G$ and $\eta \geq 0$ for the multiplier of the cap; the reduced gradients of the two legs are $g_i + \eta$ and $-g_i + \eta$. A free long leg has $g_i = -\eta$, so its short leg's reduced gradient is 2η; a long leg at its upper bound has $g_i + \eta \leq 0$, so its short leg's is at least 2η (and symmetrically). The idle leg can therefore enter only where η vanishes, at the release of the cap, and the event search drops that competing leg event so the release fires.

At the first vertex, overlapping legs (the cap tight with a zero multiplier) are removed by subtracting $\min(x_i^+, x_i^-)$ from both, which leaves w and every other constraint unchanged; a cap at the smallest feasible gross exposure ($c = 1$ under a full budget), which fails Slater's condition, is handled by solving $\min \mathbf{1}^{\mathsf{T}} x$ first and pinning every leg with a positive reduced cost.

A.3. The degenerate first vertex

The maximum-return vertex can be degenerate: every weight on a bound (any inequality row on a long-only problem whose best asset takes the whole budget), or a duplicated tight row. Its partition is then read off the linear program's duals, generalising the budget rule of §3. At a degenerate vertex these are not unique, and the solution the simplex reports can show too few zero reduced costs to free enough assets. For example, with three assets, $\mu = (3, 2, 1)$, long-only, the budget and the row $w_2 = 0$, HiGHS reports duals that free only the first asset at $w = (1, 0, 0)$, whereas the equally optimal multipliers 1 on both rows free the other two. The code therefore first moves the reported duals to a vertex of the optimal dual set, stepping along directions that keep the tight dual constraints tight until they determine the multipliers. A basis is read off that dual vertex: the assets strictly inside their bounds, then the tight rows with zero multiplier, which are released, then the assets on a bound with zero reduced cost, in order of decreasing return, which are freed. The active rows then have a square, nonsingular free block, the first segment starts at the vertex itself, and the implied multipliers are those of the dual vertex, so the partition is dual feasible. The optimal dual set has no vertex, and the vertex is refused, when a tight row touches only assets fixed at both bounds.

Nearly dependent equality rows, such as $\mathbf{1}^{\mathsf{T}} w = 1$ and $(\mathbf{1} + \epsilon v)^{\mathsf{T}} w = 1$, which encode $v^{\mathsf{T}} w = 0$

only at scale ϵ, would be enforced by the linear program only to its tolerance divided by ϵ. When the row-normalised equality matrix has a reciprocal condition number below 10^{-6} they are handed to HiGHS in orthonormal rows, $Q^{\mathsf{T}}w = R^{-\mathsf{T}}b$ with $A^{\mathsf{T}} = QR$.

B. Numerical robustness: degeneracy and ties

Round-off. A candidate turning point can be optimal to solver precision but not exactly feasible: round-off accumulated over hundreds of turning points can leave a free weight a hair outside its box. `CLA._emit` projects such a point back onto the feasible set before it validates it. The projection is a numerical repair: it keeps the point optimal to working accuracy only when the correction is small and lies in near-flat directions of the objective, where round-off usually sits, so we measure it below. For the plain budget with no active inequality row the projection onto the capped simplex is closed-form (a single shift, found by bisection, then a clip). Under a general A or an active row of G the code alternates a box clip with the affine projection onto $\{w : Cw = c\}$, $C = [\,A;\, G_{\mathcal{S}}\,]$, written in orthonormal rows so that its Gram matrix is the identity rather than CC^{T}. It usually converges in one iteration, stops once the box is violated by at most 10^{-12}, and raises a `ProjectionError` if 100 iterations do not get there.

The measurements of `make_figures.py validate-projection` support this for the problems tested. On the rank-deficiency sweep of Figure 4 and the S&P 500 windows of §7.7, the projection changes between none and four turning points per trace; the largest weight correction is 9.3×10^{-15}, the largest relative change of the objective 8.7×10^{-19}, and every projected point satisfies the box and the budget to 2.2×10^{-16}. The KKT certificate (Table 4) bounds every residual on the segments between projected points by 2×10^{-13}, and every completed trace of the sweep matches a reference QP in objective to $\sim 10^{-13}$.

Rank deficiency. The two regimes are told apart by the *conditioning* of the free block. While it stays numerically full rank any infeasibility is round-off and is projected away. Once the free set outgrows the covariance rank, $\Sigma_{\mathcal{F}\mathcal{F}}$ is singular, but the reduced system (2) need not be: the active rows $C_{\mathcal{F}}$ hold the free weights on their null space, and the system is uniquely solvable when $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$ is positive definite, where the columns of Z span $\{v : C_{\mathcal{F}}v = 0\}$. A guard reads the reciprocal condition number of $\Sigma_{\mathcal{F}\mathcal{F}}$ from its eigenvalues, a quantity that, unlike the size of a violation, is stable across BLAS builds. When it falls below 10^{-12} the guard reads that of $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$ instead, the segment is solved on the null space, and the trace is declined with a diagnosis only when that block is singular too, since projecting its output could silently yield a suboptimal frontier. Releases up to 2.5.0 declined as soon as $\Sigma_{\mathcal{F}\mathcal{F}}$ was singular, which refused the frontier of a sample covariance from fewer observations than assets whenever its free set outgrew the data rank (`make_figures.py degeneracy`). Singular constraint rows, at the first vertex or later in the trace, are declined in the same way (§3, §4.3).

Conditioning. To see the guard on its own we prescribe the spectrum, $\Sigma = Q\,\mathrm{diag}(\lambda)Q^{\top}$ with 60 eigenvalues spread geometrically from 1 to $1/\kappa$, under a long-only budget, and certify each trace (`make_figures.py validate-conditioning`). With a random orthogonal Q the

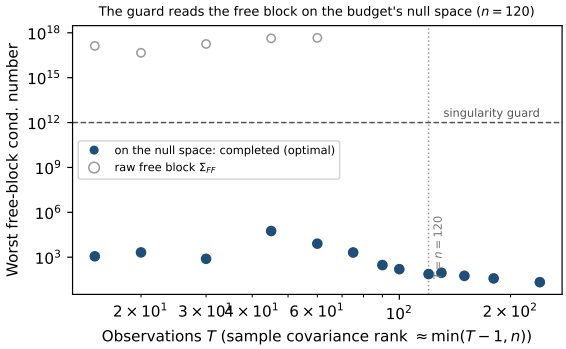


Figure 4: A rank-deficiency sweep at $n = 120$, the long-only covariance estimated from T observations. Grey rings are the worst condition number of the raw free block $\Sigma_{\mathcal{F}\mathcal{F}}$ over a trace; blue circles that of $Z^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}Z$, the block on the budget's null space, which is what the guard (dashed, 10^{12}) reads. Below $T = 75$ the raw block is singular ($\sim 10^{17}$) while the reduced block stays below 6×10^4, so every trace completes, and every completed frontier matches a reference QP to 3×10^{-13}. Release 2.3.3 declined the five windows with $T \leq 60$.

free blocks see only part of the spectrum, their worst condition number stays at 5.9×10^9 for $\kappa = 10^{16}$, and every trace completes with KKT residuals below 2.5×10^{-14}. With $Q = I$ the free block inherits the full spread: traces up to $\kappa = 10^{12}$ complete and are certified (residuals up to 5.5×10^{-12}), and from 3×10^{12} to 10^{14} every trace is declined at the guard, so here its value decides the outcome. At $\kappa = 10^{16}$ the smallest eigenvalues are zero to working precision; the trace completes again, in 10 turning points, and is certified (3.9×10^{-15}). It and **OSQP** reach different weights, up to 0.94 apart, yet both meet the optimality conditions, so there the optimum is not unique.

The reference QP is the less accurate of the two: OSQP leaves KKT residuals near 10^{-9} and differs from the trace by up to 0.2 in a weight in the diagonal family, where the KKT conditions only weakly determine the weights along directions of tiny variance. None of the 192 reference solves failed.

Event ties. Ties are handled by the least-index rule of §4.5. Across four deliberately degenerate families of 32 instances each (n up to 240; `make_figures.py tie-degeneracy`), a trace counts only if it is optimal: every turning point must pass the KKT certificate of Table 4 to 10^{-8}, so a frontier that ends but is wrong is reported as such. Exactly tied means give 31 certified traces; the other instance ($n = 60$) is declined partway along the frontier, at a degenerate turning point whose multipliers are not unique. Exchangeable pairs (identical means and covariance rows) give 32 certified traces, and the tie-break never frees both members of a pair. A family of $p \approx n/3$ group caps gives 32, with up to 53 rows active at once, and an over-constrained family of $p = n$ overlapping caps, whose maximum-return vertex is degenerate, gives 32, with up to 240 rows active at once. No trace is off the frontier.

A correction. The first version of this paper, run with release 2.3.3, counted completions without certifying them, and reported that the tied-means and exchangeable families all complete. They did complete, but none is on the efficient frontier: certified, all 32 tied-means

and all 32 exchangeable traces of 2.3.3 fail the certificate, and on the tied-means family with $n \leq 120$ their objective exceeds that of an interior-point solve by up to a factor of 26. Both families share the largest expected return between several assets, and 2.3.3 started such a frontier at an arbitrary vertex of the tied face (§3). The group-cap family was right, and 30 of the 32 overlapping-cap traces were, the other two declined.

The tied-means and group-cap families, the full-rank windows of Figure 4 and both conditioning families satisfy the hypotheses of Proposition 1 at every turning point, so for them termination is the theorem. The others do not. In the exchangeable family all 1751 tied turning points satisfy the hypotheses. The overlapping-cap traces pass 2637 ties, all with dependent constraints, so for them only the weaker guarantees of §4.5 hold. The nine rank-deficient windows of Figure 4 have a singular Σ, outside $\Sigma \succ 0$; all nine complete, the five with $T \leq 60$ by solving on the budget's null space. No trace used more than 1.5% of the iteration cap $100(n + p + 1)$.

C. Relation to other CLA implementations

The most widely used open CLA is that of Bailey and López de Prado (2013), behind **PyPortfolioOpt**'s `CLA`. It shares the greedy first turning point and the explicit minimum-variance endpoint of **cvxcla** and aims at the same turning points, but it is built without the Karush–Kuhn–Tucker conditions. Bailey and López de Prado (2013, p. 173) name them as one option for the inequality-constrained problem and instead "divide and conquer" it into unconstrained problems on the free assets. Without multipliers on the bounds, deciding which blocked asset to free takes a fresh inverse of the free block for every candidate, each inserted into closed-form Lagrange expressions for one budget row; in **cvxcla** the same decision is the sign of $\gamma_i + \lambda\delta_i$ from a single solve (§4.2). Their code then removes turning points that violate the bounds or the convex hull after the fact (`purgeNumErr`, `purgeExcess`); **cvxcla** checks the same conditions with a KKT certificate (§7.1). In **PyPortfolioOpt** 1.6.0 those violations are not round-off: after a weight leaves the free set, the next turning point is solved with that weight's previous value rather than its bound, so it misses the budget and is purged; `purgeExcess` compares expected returns without a tolerance. Both delete genuine turning points. Ties are broken by a strict comparison, with no anti-cycling rule. On the scaling problems of §7.2 **PyPortfolioOpt**'s corners are a subset of **cvxcla**'s ($19, 42, 81, 158, 323, 641$ against $21, 42, 81, 159, 323, 641$). The missing corners are genuine, so across them its frontier is wrong, by up to 7×10^{-3} in a weight at $n = 20$ and 8×10^{-5} at $n = 160$; at the other sizes the two frontiers agree to 10^{-12}, and the minimum-variance endpoints to $\sim 10^{-16}$ throughout. Reading the blocked weights from the current iterate and comparing with a tolerance removes the discrepancy.

Written with the KKT conditions, the method is also the shorter one: Figure 5 is a complete CLA for the problem class of Bailey and López de Prado (2013) in 39 lines of **NumPy**, against about 180 for the trace in **PyPortfolioOpt**. Each segment costs one bordered solve of (4), every event is read off it (§4.2), and events are compared with a relative tolerance. On the scaling problems of §7.2 it returns **cvxcla**'s turning points to 10^{-13}, in time of the same order as the dense backend. It refuses rather than mis-solves infeasible bounds, a tied largest return (§3) and a singular free block (Appendix B); **cvxcla** handles the last two.

The large-scale active-set methods of Perold (1984), Stein *et al.* (2008) and Hirschberger *et al.* (2010) cut the per-step cost through incremental factorisations, sparsity or pivoting tied to

```python
import numpy as np

def cla(mu, S, lo, up, tol=1e-9):
    """Turning points (lam, w) of max lam mu'w - w'Sw/2, 1'w=1, lo<=w<=up."""
    n, w, rest = len(mu), lo.astype(float), 1.0 - lo.sum()
    for i in np.argsort(-mu, kind="stable"):   # first turning point
        take = min(up[i] - lo[i], rest)
        w[i], rest = w[i] + take, rest - take
        if rest <= 0:
            break
    if rest > 0 or (n > 1 and np.partition(mu, -2)[-2] == mu.max()):
        raise ValueError("infeasible bounds, or a tied largest return")
    F, lam, last, path = np.arange(n) == i, np.inf, -1, [(np.inf, w.copy())]
    while True:                                # one KKT solve per segment
        B, m, one = ~F, F.sum(), np.ones(F.sum())
        K = np.block([[S[np.ix_(F, F)], one[:, None]], [one, 0.0]])
        rhs = np.c_[np.r_[-S[np.ix_(F, B)] @ w[B], 1 - w[B].sum()],
                    np.r_[mu[F], 0.0]]
        x = np.linalg.solve(K, rhs)            # w_F = a + lam b, nu likewise
        a, b = w.copy(), np.zeros(n)
        a[F], b[F] = x[:m, 0], x[:m, 1]
        gam, dlt = S @ a + x[m, 0], S @ b - mu + x[m, 1]  # reduced cost
        with np.errstate(divide="ignore", invalid="ignore"):
            hit = np.where(F, np.where(b < 0, up - a, lo - a) / b, -gam / dlt)
        live = np.where(F, abs(b) > tol,
                        np.where(w <= lo, dlt > tol, dlt < -tol) & (lo < up))
        if last >= 0:
            live[last] = False                 # the asset that just flipped
        hit = np.where(live & (hit <= lam * (1 + tol)), hit, -np.inf)
        k = int(np.argmax(hit))                # ties: lowest index
        if hit[k] <= 0:
            return [*path, (0.0, a)]
        lam, w = hit[k], a + hit[k] * b
        if np.any((w < lo - 1e-8) | (w > up + 1e-8)):
            raise np.linalg.LinAlgError("free block numerically singular")
        if F[k]:
            w[k] = up[k] if b[k] < 0 else lo[k]  # leaving weight on its bound
        F[k], last = not F[k], k
        path.append((lam, w.copy()))
```

Figure 5: A complete CLA for box bounds and one budget row, written with the KKT conditions. A runnable copy that checks it against **cvxcla** is at https://gist.github.com/tschm/a622ba482795b4a9fc452c64e1024982.

an explicit matrix; we could not run their codes or that of Niedermayer and Niedermayer (2010) on current toolchains, and compare instead with **qpOASES** (§7.6).

D. Proof of Proposition 1

Leaving is permanent. A partition (free set, blocked bounds, active rows $\mathcal{S}$) is valid at λ when its segment respects the bounds and the inactive rows and its multipliers have the

right signs. Each of these conditions is affine in λ. An event fires at $\lambda^\star$ because one of them is zero there with a slope that violates it below $\lambda^\star$ (the slope floor makes every event a strict crossing), and an affine function that crosses zero stays on the wrong side, so a partition left at $\lambda^\star$ is invalid at every $\lambda < \lambda^\star$. Since λ never increases, a partition can recur only at the same $\lambda^\star$, within a tie.

The direction problem at a tie. Fix $\lambda^\star$. Because $\Sigma \succ 0$ the optimiser $w^\star = w(\lambda^\star)$ is unique, and by linear independence so are its multipliers. Constraints with slack, or holding with a nonzero multiplier, cannot fire at $\lambda^\star$, and every partition the walk visits there keeps their status; let E stack the gradients of the latter together with A. The partitions differ only on the set T of tied constraints, those holding with a zero multiplier, written $c_t^\mathsf{T} w \le \text{const}$ (with $c_t = \pm e_i$ for a bound). Below $\lambda^\star$ the walk follows a segment $w(\lambda^\star - \tau) = w^\star + \tau d$, $d = -\beta$, on which the multipliers are affine in τ too, $\nu(\tau) = \nu^\star + \tau\nu'$. Substitute both into the KKT conditions of (1) and keep the terms of order τ. Stationarity gives $\Sigma d + \mu + E^\mathsf{T}\nu'_E + C_T^\mathsf{T}\nu'_T = 0$. A constraint in E keeps a nonzero multiplier for small τ, so it stays active and $Ed = 0$. A tied constraint has slack $-\tau\, c_t^\mathsf{T} d$ and multiplier $\tau\,\nu'_t$, so feasibility, the sign of the multiplier, and complementary slackness require $c_t^\mathsf{T} d \le 0$, $\nu'_t \ge 0$, and $(c_t^\mathsf{T} d)\,\nu'_t = 0$. These are the KKT conditions of the strictly convex QP

$$\min_d \ \tfrac{1}{2}\, d^\mathsf{T}\Sigma d + \mu^\mathsf{T} d \quad \text{subject to} \quad Ed = 0, \quad c_t^\mathsf{T} d \le 0 \ \ (t \in T),$$

so d is its unique solution, the classical directional derivative of a parametric program without strict complementarity (Jittorntrum 1984; Bonnans and Shapiro 2000).

The complementarity problem. Let E have linearly independent rows, let Z be a basis of its null space, and let C_T collect the columns c_t, $t \in T$. Every d with $Ed = 0$ is $d = Zy$, and the direction problem becomes $\min_y \tfrac{1}{2}\, y^\mathsf{T} H y + \mu^\mathsf{T} Z y$ subject to $C_T^\mathsf{T} Z y \le 0$, with $H = Z^\mathsf{T}\Sigma Z \succ 0$ because $\Sigma \succ 0$ and Z has full column rank. Its KKT conditions, with slacks $s = -C_T^\mathsf{T} Z y$ and multipliers $\nu \ge 0$, eliminate $y = -H^{-1}Z^\mathsf{T}(\mu + C_T\nu)$ and leave the linear complementarity problem

$$s = q + M\nu, \qquad s \ge 0,\ \nu \ge 0,\ s^\mathsf{T}\nu = 0, \qquad M = C_T^\mathsf{T} Z H^{-1} Z^\mathsf{T} C_T, \quad q = C_T^\mathsf{T} Z H^{-1} Z^\mathsf{T}\mu.$$

M is positive semidefinite, and positive definite exactly when $Z^\mathsf{T} C_T$ has full column rank. Suppose $Z^\mathsf{T} C_T\, x = 0$. Then $C_T x$ lies in the null space of Z^T, which is the row space of E, so $C_T x = E^\mathsf{T} z$ for some z, and $[\,E^\mathsf{T},\ C_T\,]\,(-z, x) = 0$. The hypothesis that the gradients of E and of the tied constraints are linearly independent forces $z = 0$ and $x = 0$. Hence $M \succ 0$, and in particular M is a P-matrix: every principal minor is positive.

The correspondence with the implementation. Each tied constraint t contributes one complementary pair (s_t, ν_t), and a partition visited at $\lambda^\star$ makes exactly one of the two basic: s_t when the constraint is released (a free weight, an inactive row), ν_t when it is enforced (a blocked weight, an active row). The β-system of that partition (§4.1) is the equality-constrained form of the direction problem with the enforced constraints held, so its solution is the basic solution of the complementarity problem. Table 5 lists the six event types of the code (`_events.py`). Each fires at $\lambda^\star$ exactly when the basic variable of its pair is negative: a released constraint whose slack would turn negative as λ falls, or an enforced one whose multiplier would.

Event (column)	Status of t	Basic variable	Fires at $\lambda^\star$ when
free weight reaches its upper bound (0)	released	s_t	$\beta_i < 0$
free weight reaches its lower bound (1)	released	s_t	$\beta_i > 0$
blocked weight leaves its upper bound (2)	enforced	ν_t	$\delta_i < 0$
blocked weight leaves its lower bound (3)	enforced	ν_t	$\delta_i > 0$
inactive row binds (0, row block)	released	s_t	$g_j^{\mathsf{T}}\beta < 0$
active row releases (1, row block)	enforced	ν_t	$\eta_j^{\beta} > 0$

Table 5: The six event types of the event scan and the complementary pair each one pivots. A coordinate's status admits one event type at a time, so a coordinate index identifies the pair.

The pivoting rule. When several events fire at $\lambda^\star$, `select_next_event` (`pathtracer.py`) flips the one with the lowest coordinate index, assets before inequality rows, and changes the status of that coordinate alone. By the correspondence above this is a simple principal pivot on the lowest-index pair whose basic variable is negative, in a fixed ordering of the pairs: Murty's least-index method (Murty 1974), which terminates on every linear complementarity problem with a P-matrix, for every q (Murty 1974; Cottle, Pang, and Stone 1992). The pivot depends only on the current partition, so a partition repeated at $\lambda^\star$ would repeat forever; none is, and the walk leaves $\lambda^\star$ after finitely many steps with a feasible basic solution, along a segment of positive length.

Termination. Each segment of positive length uses a partition never used before, and there are finitely many partitions; each tie between two such segments takes finitely many steps.

References

Andersson JAE, Gillis J, Horn G, Rawlings JB, Diehl M (2019). "CasADi: a software framework for nonlinear optimization and optimal control." *Mathematical Programming Computation*, **11**(1), 1–36.

Bailey DH, López de Prado M (2013). "An open-source implementation of the critical-line algorithm for portfolio optimization." *Algorithms*, **6**(1), 169–196.

Best MJ (1996). "An algorithm for the solution of the parametric quadratic programming problem." In H Fischer, B Riedmüller, S Schäffler (eds.), *Applied Mathematics and Parallel Computing*, pp. 57–76. Physica-Verlag, Heidelberg.

Best MJ (2010). *Portfolio Optimization*. Chapman & Hall/CRC, Boca Raton, FL.

Bonnans JF, Shapiro A (2000). *Perturbation Analysis of Optimization Problems*. Springer-Verlag, New York.

Cajas D (2021). "Riskfolio-Lib: portfolio optimization and quantitative strategic asset allocation in Python." `https://github.com/dcajasn/Riskfolio-Lib`.

Chamberlain G, Rothschild M (1983). "Arbitrage, factor structure, and mean–variance analysis on large asset markets." *Econometrica*, **51**(5), 1281–1304.

Cottle RW, Pang JS, Stone RE (1992). *The Linear Complementarity Problem*. Academic Press, Boston, MA.

Fan J, Liao Y, Mincheva M (2013). "Large covariance estimation by thresholding principal orthogonal complements." *Journal of the Royal Statistical Society: Series B*, **75**(4), 603–680.

Ferreau HJ, Kirches C, Potschka A, Bock HG, Diehl M (2014). "qpOASES: a parametric active-set algorithm for quadratic programming." *Mathematical Programming Computation*, **6**(4), 327–363.

Fu A, Narasimhan B, Boyd S (2020). "CVXR: an R package for disciplined convex optimization." *Journal of Statistical Software*, **94**(14), 1–34.

Hirschberger M, Qi Y, Steuer RE (2010). "Large-scale MV efficient frontier computation via a procedure of parametric quadratic programming." *European Journal of Operational Research*, **204**(3), 581–588.

Jittorntrum K (1984). "Solution point differentiability without strict complementarity in nonlinear programming." *Mathematical Programming Study*, **21**, 127–138.

Johnstone IM (2001). "On the distribution of the largest eigenvalue in principal components analysis." *The Annals of Statistics*, **29**(2), 295–327.

Laloux L, Cizeau P, Bouchaud JP, Potters M (1999). "Noise dressing of financial correlation matrices." *Physical Review Letters*, **83**(7), 1467–1470.

Ledoit O, Wolf M (2004). "A well-conditioned estimator for large-dimensional covariance matrices." *Journal of Multivariate Analysis*, **88**(2), 365–411.

Markowitz H, Starer D, Fram H, Gerber S (2021). "Avoiding the downside: A practical review of the critical line algorithm for mean–semivariance portfolio optimization." In *Handbook of Applied Investment Research*, chapter 17. World Scientific. doi:10.1142/9789811222634_0017. URL https://doi.org/10.1142/9789811222634_0017.

Markowitz HM (1952). "Portfolio selection." *The Journal of Finance*, **7**(1), 77–91.

Markowitz HM (1956). "The optimization of a quadratic function subject to linear constraints." *Naval Research Logistics Quarterly*, **3**(1–2), 111–133.

Markowitz HM (1959). *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, New York.

Markowitz HM, Todd GP (2000). *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Frank J. Fabozzi Associates, New Hope, PA.

Martin RA (2021). "PyPortfolioOpt: portfolio optimization in Python." *Journal of Open Source Software*, **6**(61), 3066. doi:10.21105/joss.03066. URL https://doi.org/10.21105/joss.03066.

Murty KG (1974). "Note on a Bard-type scheme for solving the complementarity problem." *Opsearch*, **11**(2–3), 123–130.

Nicolini C, Manzi M, Delatte H (2025). "skfolio: Portfolio optimization in Python." *arXiv preprint arXiv:2507.04176*.

Niedermayer A, Niedermayer D (2010). "Applying Markowitz's critical line algorithm." In *Handbook of Portfolio Construction*, pp. 383–400. Springer.

Perold AF (1984). "Large-scale portfolio optimization." *Management Science*, **30**(10), 1143–1160.

Schmelzer T (2026a). "The Constrained LASSO is a Portfolio: A Numerical Check of the CLA–LASSO Identity under Constraints." *Journal of Reproducible Statistics*, **1**, 10. doi:10.68365/jrepstat.2026.10. URL https://doi.org/10.68365/jrepstat.2026.10.

Schmelzer T (2026b). "The Efficient Frontier from a LASSO Solver: A Numerical Check of the CLA–LASSO Identity." *Journal of Reproducible Statistics*, **1**, 7. doi:10.68365/jrepstat.2026.7. URL https://doi.org/10.68365/jrepstat.2026.7.

Schmelzer T (2026c). ***cvxcla**: the Critical Line Algorithm with a quadratic-form operator interface*. Python package version 2.7.1, archived on Zenodo; source at https://github.com/cvxgrp/cvxcla. TODO: the Zenodo DOI of this release.

Schmelzer T, Hastie T (2026). "The Critical Line Algorithm and the Constrained LASSO: One Curve, Two Literatures." *arXiv preprint arXiv:2609.25704*.

Schmelzer T, Stoll M (2026). "Non-negative conjugate gradients." *arXiv preprint arXiv:2607.22121*.

Sharpe WF (1963). "A simplified model for portfolio analysis." *Management Science*, **9**(2), 277–293.

Stein M, Branke J, Schmeck H (2008). "Efficient implementation of an active set algorithm for large-scale portfolio selection." *Computers & Operations Research*, **35**(12), 3945–3961.

Stellato B, Banjac G, Goulart P, Bemporad A, Boyd S (2020). "OSQP: An Operator Splitting Solver for Quadratic Programs." *Mathematical Programming Computation*, **12**(4), 637–672.

Wolfe P (1959). "The simplex method for quadratic programming." *Econometrica*, **27**(3), 382–398.

Würtz D, Chalabi Y, Chen W, Ellis A (2009). *Portfolio Optimization with R/Rmetrics*. Rmetrics Association and Finance Online. R package `fPortfolio`.

Affiliation:

Thomas Schmelzer
Jebel Quant Research
Abu Dhabi, United Arab Emirates
E-mail: thomas@jqr.ae
URL: https://github.com/cvxgrp/cvxcla
Preprint: doi:10.2139/ssrn.7583179